

Contents

Phase 1 — Graph Profiling	1
1. The goal	1
2. Setup: tracing the training step	1
3. The profiler — component reference	2
3.1 High-level idea	2
3.2 The flow	2
3.3 Data types	3
3.4 Tensor inspection helpers	3
3.5 Aliasing detection	4
3.6 <code>GraphProfiler</code> — static passes	6
3.7 <code>GraphProfiler</code> — runtime	8
3.8 <code>GraphProfiler</code> — queries	8
3.9 Note on what “static” means	9
4. The three real challenges	10
4.1 Placeholder lifetime	10
4.2 Aliasing — the central abstraction	10
4.3 Per-element role attribution (the <code>conv_backward</code> trap)	11
5. Results	12
6. Known limitations	13
7. Files	13

Phase 1 — Graph Profiling

A self-contained walk-through of what we built, why we built it that way, and the surprises we hit along the way.

1. The goal

Training a neural network on a GPU spends most of its memory on things you do not directly think about: forward activations that have to stay alive until the backward pass uses them, gradient buffers, optimizer state. If you want to *reduce* that memory (Phase 2 / 3 do this with activation checkpointing) you first have to *see* it.

Phase 1 is that “seeing” tool — a static profiler that, given one model and batch size, produces:

1. **A stacked-area chart** of live GPU memory over the course of one training iteration, broken down by tensor role (parameters / activations / gradients / other). One plot per (`model`, `batch_size`).
2. **A bar chart** of peak memory across batch sizes, one per model.

Plus per-iteration latency, per-node timing, and the list of intermediate activations and their lifetimes — everything Phase 2 will need.

2. Setup: tracing the training step

A training iteration is four phases glued end-to-end:

[FORWARD] → [LOSS] → [BACKWARD] → [OPTIMIZER STEP]

We use PyTorch FX (`make_fx` via `graph_tracer.compile`) to trace one invocation of `train_step(model, optim, inputs)` into a flat `fx.GraphModule` containing every primitive op (`matmul`, `convolution`, `add`, `_foreach_add`, etc.) PyTorch would execute. Two custom “separator” ops, `sep` and `sep_backward`,

are inserted by `SEPFunction` in `graph_tracer.py` so the profiler can locate the forward/loss and backward/optimizer boundaries inside the flat op list.

The result is a graph of N nodes (N ranges from ~800 for the dummy MLP to ~8800 for BERT-base). Each node has:

- an `op` (placeholder, `call_function`, output)
- a `target` (the ATen op or builtin)
- `args` and `all_input_nodes` (its inputs)
- `users` (its consumers)
- `meta["val"]` — a `FakeTensor` describing the output's shape and dtype

That metadata is everything we need. We never execute the graph to compute the memory chart; the chart comes from analyzing this object.

3. The profiler — component reference

`GraphProfiler` (in `graph_prof.py`) subclasses `torch.fx.Interpreter`. The rest of this section walks every piece of the file in the order it appears, explaining **why we need it**, **the idea**, and **how it flows into the next thing**.

3.1 High-level idea

The profiler turns one traced training step (a flat list of ~800–8800 FX nodes) into:

1. a memory chart by tensor role over the iteration, and
2. per-node timing data Phase 2 will use.

Doing this requires answering, for every node: *what region am I in? what role does my output play? how many bytes do I produce? how long do I live? whose storage do I share?* Once those questions are answered statically (without executing the model), the chart is a one-pass sweep over the answers.

3.2 The flow

```
GraphProfiler(gm)           # constructor runs 6 static passes
    find_separators         # locate phase boundaries
    assign_regions          # label every node by phase
    classify_tensors         # PARAM / GRAD / OPT_STATE / OTHER
    find_intermediates      # add ACT for forward outputs read in backward
    compute_sizes           # per-node bytes
    compute_aliases         # storage owner for every node
    reset_stats             # zero out runtime accumulators

profiler.run(*args)         # warm-up: time every op with CUDA events
profiler.reset_stats()      # discard warm-up samples
profiler.run(*args) × 3     # measurement: same, but kept
profiler.aggregate_stats()  # average across the 3 runs

profiler.memory_timeline_by_role() # for plotting
profiler.peak_memory_bytes()      # for the bar chart
profiler.iteration_latency_ms()   # for the latency table
profiler.print_summary()          # console
```

Static state is built once in `__init__` and never recomputed; runtime state is appended to per `run()` call and averaged in `aggregate_stats`.

3.3 Data types

NodeType — the role of a tensor

```
class NodeType(IntEnum):
    OTHER      = 0
    OPT_STATE  = 1
    ACT        = 2
    GRAD       = 3
    PARAM      = 4
```

Why: the chart’s whole point is to show *which kind* of bytes are on the GPU. Without a role label every byte would be a meaningless gray slab.

Idea: an `IntEnum` so the values do double duty as merge priority. When a single storage allocation is shared by several aliases that disagree on role (a parameter that’s also viewed in backward, say), `max(role_a, role_b)` gives us the most-important label automatically — PARAM beats GRAD beats ACT beats OPT_STATE beats OTHER.

Flow: written by `classify_tensors`, read by `memory_timeline_by_role` when summing bytes per bucket.

Region — which phase of the iteration a node belongs to

```
class Region(Enum):
    FORWARD, LOSS, BACKWARD, OPTIMIZER
```

Why: the chart marks the `sep` / `sep_backward` / `optimizer` boundaries as vertical guides; classification needs to know “is this op in the backward pass?” to decide if it’s producing a gradient.

Flow: written by `assign_regions`, used implicitly by `classify_tensors` (via `sep_idx`, `sep_bwd_idx`, `opt_idx`) and displayed by `print_summary`.

Intermediate — a forward activation kept for backward

```
@dataclass
class Intermediate:
    node:          fx.Node
    size_bytes:     int
    last_fwd_idx:   int
    first_bwd_idx:  int
    recompute_ms:   float
```

Why: this is the data structure Phase 2 (activation checkpointing) selects from. Each field is exactly what the μ -TWO greedy algorithm needs to score an eviction candidate (`size` / `recompute_ms` ratio, plus the lifetime gap between `last_fwd_idx` and `first_bwd_idx`).

Flow: populated by `find_intermediates` (sets node, lifetime indices) and `compute_sizes` (back-fills `size_bytes`); `recompute_ms` is filled in `aggregate_stats` from the per-node CUDA timings.

DISPLAY_ROLES

```
DISPLAY_ROLES = (PARAM, OPT_STATE, ACT, GRAD, OTHER)
```

Why: `NodeType`’s iteration order is OTHER, OPT_STATE, ACT, GRAD, PARAM (definition order, low value to high) — fine for `max()` but backwards for human reading. We want PARAM at the top of printed tables since it’s the persistent baseline.

3.4 Tensor inspection helpers

iter_tensors(val) — flatten any nested structure to its tensors

```
def iter_tensors(val):
    if isinstance(val, torch.Tensor): yield val
    elif isinstance(val, (list, tuple)):
        for v in val: yield from iter_tensors(v)
```

Why: a node’s `meta["val"]` can be a `Tensor`, a `list[Tensor]`, a `tuple[Tensor, ...]`, or even a nested combination (some ops return `(Tensor, list[Tensor])`). We need a uniform way to walk it.

Flow: used by `output_bytes` and `contains_tensor`.

`contains_tensor(val)`

```
def contains_tensor(val):
    return next(iter_tensors(val), None) is not None
```

Why (subtle but important): we need to ask “does this list contain at least one tensor?” The intuitive `any(iter_tensors(val))` *crashes under FakeTensor* — `any` calls `bool(t)` on each element, and `FakeTensor` refuses (`bool` would force materializing the tensor’s data, which a fake tensor doesn’t have). Using `next(..., None)` only checks if there’s an element to yield, never inspects its truthiness.

Flow: called by `is_container`.

`output_bytes(node)` — logical size of a node’s output

```
def output_bytes(node):
    if node.op not in ("call_function", "placeholder"): return 0
    return sum(t.numel() * t.element_size()
               for t in iter_tensors(node.meta.get("val")))
```

Why: the raw “what shape and dtype is this op producing” answer. For a single-tensor output it’s `numel * element_size`; for multi-output ops it’s the sum across all returned tensors.

Flow: used both directly (`compute_sizes` stores it as `node_logical_size_bytes`) and inside `allocation_bytes`.

3.5 Aliasing detection

These helpers are what lets the profiler avoid double-counting. They work in concert with `compute_aliases` (§3.6).

`alias_in_schema(node)` — schema says output aliases input Already covered in detail in §4.2 (Pattern A). Returns `True` for in-place / view / fused ops whose ATen schema marks any return as aliasing an input.

`is_getitem(node)` — is this a list/tuple subscript?

```
def is_getitem(node):
    return (node.op == "call_function"
            and (node.target is operator.getitem
                 or getattr(node.target, "__name__", None) == "getitem"))
```

Why: every multi-output op (`_foreach_div`, `aten.split`, `convolution_backward`) is unpacked in FX via explicit `operator.getitem` nodes — there’s no `a, b, c = foo()` syntax in the graph. `is_getitem` is the primitive that detects them.

Idea: two-clause check — `target is operator.getitem` for the normal case; `__name__ == "getitem"` as a defensive fallback for when an FX pass wraps the target and breaks identity comparison.

Flow: used by `is_getitem_of_container` (3 lines below) and `allocation_bytes` (the `all(is_getitem(u) for u in node.users)` check).

`is_container(node)` and `is_getitem_of_container(node)`

```
def is_container(node):
    val = node.meta.get("val")
    return isinstance(val, (list, tuple)) and contains_tensor(val)

def is_getitem_of_container(node):
    return (is_getitem(node) and bool(node.all_input_nodes)
            and is_container(node.all_input_nodes[0]))
```

Why: distinguishes the two patterns that drive multi-output handling. `is_container` flags any node whose output is a tensor list/tuple (the *parent* of a `getitem` chain). `is_getitem_of_container` flags any `getitem` whose parent is such a container — the alias-chaining trigger.

Flow: `is_container` filters parents in `allocation_bytes`; `is_getitem_of_container` is the entry condition for the `getitem` branch in `compute_aliases`.

`allocation_bytes(node)` — bytes of NEW storage

```
def allocation_bytes(node):
    if node.op == "call_function":
        if alias_in_schema(node): return 0
        if is_container(node) and all(is_getitem(u) for u in node.users):
            return 0
    if is_getitem_of_container(node):
        parent = node.all_input_nodes[0]
        if parent.op == "call_function" and alias_in_schema(parent):
            return 0
    return output_bytes(node)
```

Why: this is the function that decides “do I count this node’s bytes, or are they already counted somewhere else?” Every node either allocates fresh storage (return `output_bytes`) or just references existing storage (return 0).

Idea: three early-exit cases, then fall through to “yes, count it”:

1. **alias in schema** (in-place / view): the op handed back a reference.
2. **container with only `getitem` users** (e.g., `conv_backward`): the parent doesn’t own bytes; the *getitem*s do. This is what makes per-element role attribution work — see §4.3.
3. **getitem of an aliasing container** (e.g., `getitem(foreach_mul_, i)`): the `getitem` extracts a view of one of the parent’s inputs; no new bytes.

Flow: called for every node in `compute_sizes` and stored as `node_size_bytes`. This is the value the live-memory walk sums.

`alias_target(parent, getitem)` — which input element backs this `getitem`?

```
def alias_target(parent, getitem):
    if len(getitem.args) < 2 or not parent.args: return None
    first = parent.args[0]
    if isinstance(first, fx.Node): return first
    idx = getitem.args[1]
    if (isinstance(first, (list, tuple)) and isinstance(idx, int)
        and 0 <= idx < len(first) and isinstance(first[idx], fx.Node)):
        return first[idx]
    return None
```

Why: when an aliasing multi-output op’s outputs are unpacked, each `getitem` corresponds to a *specific* input element, not just “the first one.” For `_foreach_mul_(a_list, b)` with 62 elements, `getitem #5` aliases

`a_list[5]`, not `a_list[0]`. Without per-element resolution the chain breaks (we’d attribute everything to the first element and under-count the rest — exactly the bug that caused BERT’s optimizer spike, see §4.3).

Idea: handle two patterns: - single-input multi-output (`aten.split(x, sizes) → all outputs view x`): return `args[0]` regardless of index. - list-input multi-output (`_foreach_mul_(a_list, b) → output[i]` aliases `a_list[i]`): return `args[0][i]`.

Flow: called by `compute_aliases` when wiring a getitem to its storage owner.

3.6 GraphProfiler — static passes

`__init__(gm)`

```
def __init__(self, gm):
    super().__init__(gm)
    self.nodes = list(self.module.graph.nodes)
    self.idx = {n: i for i, n in enumerate(self.nodes)}
    self.find_separators()
    self.assign_regions()
    self.classify_tensors()
    self.find_intermediates()
    self.compute_sizes()
    self.compute_aliases()
    self.reset_stats()
```

Why: single entry point. The user does `GraphProfiler(gm)` and gets a fully analyzed object.

Idea: cache the node list in topological order and a node→index map up front (everything else uses these), then run the six static passes in dependency order.

`find_separators()`

```
for i, n in enumerate(self.nodes):
    if n.op != "call_function": continue
    t = n.target
    if t == sep.default: self.sep_idx = i
    elif t == sep_backward.default: self.sep_bwd_idx = i
    elif t == aten._fused_adam.default: self.opt_idx = i
    elif "_foreach_" in str(t) and self.sep_bwd_idx >= 0 and first_foreach_after_bwd < 0:
        first_foreach_after_bwd = i
if self.opt_idx < 0: self.opt_idx = first_foreach_after_bwd
```

Why: every other pass needs to know “where does forward end and backward start?” `SEPPFunction` (in `graph_tracer.py`) inserts the markers; this pass just finds their indices.

Idea: one pass over the nodes, recording four indices. The optimizer index is either the explicit fused-Adam call (if the model uses `fused=True`) or the first `_foreach_*` op after `sep_backward` (if it uses `foreach=True`).

Flow: `sep_idx`, `sep_bwd_idx`, `opt_idx` consumed by every later pass and by `assign_regions`.

`assign_regions()`

```
for i, n in enumerate(self.nodes):
    if 0 <= self.opt_idx <= i: self.region[n] = Region.OPTIMIZER
    elif i >= self.sep_bwd_idx: self.region[n] = Region.BACKWARD
    elif i > self.sep_idx: self.region[n] = Region.LOSS
    else: self.region[n] = Region.FORWARD
```

Why: nice human-readable label per node, used for the chart’s boundary lines and for `print_summary`’s region counts.

Idea: simple if/elif from highest-index region down. The order matters — once we know we’re past `opt_idx` we’re in `OPTIMIZER` no matter what.

classify_tensors() **Why:** assigns the `PARAM`, `GRAD`, `OPT_STATE`, `OTHER` portion of the role labels (`ACT` is added next, in `find_intermediates`). These labels drive the colored bands in the chart.

Idea: two paths depending on which optimizer style was traced:

- *Fused-Adam path* (`aten._fused_adam`): the op’s args literally spell out the lists — `args[0]` is params, `args[1]` is grads, `args[2:]` are the optimizer-state tensors (`m`, `v`, `max_v`, `step`). Read them off directly.
- *Foreach path*: no single op enumerates everything. We infer:
 - **PARAM** = a placeholder used in *both* the forward and the optimizer (i.e., a learnable weight).
 - **OPT_STATE** = a placeholder used only in the optimizer (Adam’s `exp_avg`, `exp_avg_sq`, step counters).
 - **GRAD** = a backward call_function whose output flows directly into an optimizer op.

Flow: builds `self.params`, `self.grads`, `self.opt_states`, then combines them into `self.node_type`: `Dict[Node, NodeType]` (everything else falls through to `OTHER`).

find_intermediates()

```
for n in self.nodes:
    if i >= self.sep_idx or n.op != "call_function" or n in self.params: continue
    user_idxs = [self.idx.get(u, -1) for u in n.users]
    if not any(j >= self.sep_bwd_idx for j in user_idxs): continue
    self.intermediates.append(Intermediate(...))
    self.node_type[n] = NodeType.ACT
```

Why: the activations Phase 2 chooses between are exactly the forward `call_function` outputs whose users include something in the backward pass — i.e., values produced by forward but kept around to help compute the backward. Anything used only in forward can be freed right away and isn’t a candidate for AC.

Idea: filter to forward `call_functions`; check that *some* user is beyond `sep_bwd_idx`; record both the last forward use (for the “forward window”) and the first backward use (for the “backward window”) so Phase 2’s simulator can trace each one’s lifetime. Promotes the role from `OTHER` to `ACT`.

Flow: writes `self.intermediates` (read by Phase 2 and `compute_sizes`) and updates `self.node_type` (read by `memory_timeline_by_role`).

compute_sizes()

```
self.node_logical_size_bytes = {n: output_bytes(n) for n in self.nodes}
self.node_size_bytes = {n: allocation_bytes(n) for n in self.nodes}
for inter in self.intermediates:
    inter.size_bytes = self.node_logical_size_bytes[inter.node]
```

Why: precompute both byte counts for every node so later passes and queries can read them in $O(1)$.

Idea: two parallel dicts. *Logical* size is the raw output bytes (useful for Phase 2’s eviction scoring — what we’d save by recomputing this exact tensor). *Allocation* size is what the live-memory walk needs (what the GPU actually allocates that isn’t already counted via some other node’s storage).

Flow: read by `compute_aliases` (which uses the logical size to pick first-sized inputs) and by `memory_timeline_by_role` (which uses the allocation size to bucket bytes).

```

compute_aliases()
for n in self.nodes:
    owner = n
    if is_getitem_of_container(n):
        parent = n.all_input_nodes[0]
        if parent.op == "call_function" and alias_in_schema(parent):
            target = alias_target(parent, n)
            if target is not None and self.node_logical_size_bytes.get(target, 0) > 0:
                owner = self.storage_owner.get(target, target)
    elif n.op == "call_function" and alias_in_schema(n):
        for inp in n.all_input_nodes:
            if self.node_logical_size_bytes.get(inp, 0) > 0:
                owner = self.storage_owner.get(inp, inp)
                break
    self.storage_owner[n] = owner
    self.aliases_by_owner[owner].append(n)

```

Why: the central trick. Without this, our memory walk would double-count every aliased tensor. This pass collapses each chain of aliases into a single *owner* — the one node that “really” owns the GPU storage.

Idea: for every node, ask “do I alias something?”: - If I’m a getitem of an aliasing container, my owner is the input element that backs me (via `alias_target`). - If my schema says I alias one of my inputs, my owner is the first sized input (which is the in-place target for `_foreach_*`, the source for views). - Otherwise I’m my own owner. Walk in topological order so `storage_owner.get(target, target)` already has chained owners resolved when we look them up.

Flow: produces `storage_owner: Node → Node` and `aliases_by_owner: Node → List[Node]`. `memory_timeline_by_role` iterates the latter.

3.7 GraphProfiler — runtime

`reset_stats()` / `run()` / `run_node()` / `aggregate_stats()` **Why:** Phase 2 needs per-node runtime to compute the `size / recompute_ms` ratio that drives the μ -TWO greedy algorithm. We collect it via CUDA events on a few measurement runs.

Idea: standard pattern — two warm-up iterations to let cuDNN’s algorithm picker and the caching allocator stabilize, then `reset_stats()` to discard those samples, then three measurement iterations. Each `run_node` brackets the inherited dispatch with `cuda.Event.record()` calls; `run` does the same at the iteration level. `aggregate_stats` averages across samples and back-fills `Intermediate.recompute_ms` so Phase 2 has it.

Flow:

```

profiler.run(...) × 2          # warm-up
profiler.reset_stats()
profiler.run(...) × 3          # measurement
profiler.aggregate_stats()

```

The runtime accumulators (`_runtimes_ms`, `_latencies_ms`, `_memory_after_bytes`, `_memory_delta_bytes`, `_memory_peak_bytes`) are private; their averaged versions (`avg_runtime_ms`, `avg_iter_latency_ms`, ...) are the public reads.

3.8 GraphProfiler — queries

`memory_timeline_by_role()` — the heart of the chart

```

n = len(self.nodes)
timeline = {nt: [0] * n for nt in NodeType}
for owner, aliases in self.aliases_by_owner.items():

```



```

size = self.node_size_bytes.get(owner, 0)
if size == 0: continue
role = max(self.node_type[a] for a in aliases)
if owner.op == "placeholder":
    lo, hi = 0, n - 1
else:
    lo = self.idx[owner]
    hi = lo
    for a in aliases:
        hi = max(hi, self.idx[a],
                  *(self.idx[u] for u in a.users if u in self.idx))
for t in range(lo, min(hi + 1, n)):
    timeline[role][t] += size
return timeline

```

Why: produces the per-step, per-role byte counts that the stacked area chart plots.

Idea: walk *by owner* (not by node). Each owner contributes its size *once* over the closed interval [lo, hi]. Two cases: - **Placeholder owners** (params, opt state, batched inputs) are live for the *whole* iteration — they exist before step 0 and persist after step N-1. See §4.1 for why this special case matters. - **Non-placeholder owners** are live from their production index through the maximum index of any alias or any alias’s user. The union over aliases captures the full extent of the storage chain (e.g., $\text{denom} = \text{sqrt}(v) \rightarrow \text{mul}(\text{denom}) \rightarrow \text{add}(\text{denom}) \rightarrow \text{addcdiv}_()$).

The role assigned is $\max(\text{role across aliases})$, taking advantage of the IntEnum ordering so that the strongest-priority role wins.

Flow: read by `peak_memory_bytes`, `print_summary`, and the visualizer.

`peak_memory_bytes()` and `iteration_latency_ms()`

```

def peak_memory_bytes(self):
    roles = self.memory_timeline_by_role()
    return max(sum(roles[nt][t] for nt in NodeType) for t in range(n))

def iteration_latency_ms(self):
    return self.avg_iter_latency_ms

```

Why: the two scalar headline numbers — peak memory (for the bar chart) and iteration latency (for the consolidated table).

Idea: peak is just $\max$ over the timeline; latency is a one-line getter.

`print_summary()` **Why:** a one-screen console report so the user can sanity-check each run as it happens.

Idea: count nodes by role and by region (using `Counter`), find the peak step, print a small table with one row per role plus a TOTAL, and the iteration latency.

3.9 Note on what “static” means

Every quantity in §3.6’s static passes is **exact**, not an estimate. `output_bytes` reads `meta["val"]`, which carries the exact shape and dtype the op produces; `numel * element_size` is precisely what the GPU allocator will ask for. Lifetimes come from `node.users` plus topological order — a deterministic property of the graph. Aliasing comes from the ATen schema, which authoritatively says which ops mutate inputs vs. allocate fresh storage.

So `peak_memory_bytes()` is an exact count of the bytes held by **FX-graph-visible tensors** at the peak step. It’s smaller than what the GPU actually uses during execution by exactly the size of memory that

exists *because the model runs*, not because of any property of the graph: cuDNN/cuBLAS workspaces a kernel grabs internally, op- internal scratch buffers freed before the kernel returns. These are real `cudaMalloc` calls but they aren't the output of any FX node, so the static walk can't see them. We acknowledge this honestly and don't try to attribute it.

4. The three real challenges

Each one bit us during development; documenting them so the choices in the code are explicable.

4.1 Placeholder lifetime

Symptom: on ResNet18, the static walk reported 0.04 MB live at step 0 and 42 MB at the last step — nowhere near the ~143 MB the model + Adam moments actually take.

Cause: a placeholder is one specific node at index `i` in the FX graph (e.g., `arg0_42` is “the 42nd parameter of the model”). The naive walk says it's live from `idx[arg0_42]` to `max(idx of users)`. But all placeholders are passed in *simultaneously* at the start of `run()`, and they're held by `model` and `optim` Python objects *across* iterations. Their physical lifetime is **the entire iteration**, not just from their node index to the optimizer step that last touches them.

Fix: in `memory_timeline_by_role`, special-case placeholders to be live from `lo=0` to `hi=N-1`. After the fix, ResNet18's chart shows a flat ~143 MB baseline (PARAM + opt-state) running across the whole iteration, with the activation hump on top.

4.2 Aliasing — the central abstraction

This is the single most important idea in the profiler. Three FX patterns make several distinct nodes share one underlying allocation, and counting each node naively double-counts its bytes (or worse — N-times-counts on a 62-element foreach).

Pattern A — schema aliasing (in-place / view ops) Examples: `relu_`, `view`, `transpose`, `_foreach_add_`, `_fused_adam`. These ops *don't allocate any new GPU memory* — they just hand back a reference to memory that already exists. Detecting them is what `alias_in_schema` does:

```
def alias_in_schema(node: fx.Node) -> bool:
    schema = getattr(node.target, "_schema", None)
    return schema is not None and any(r.alias_info is not None
                                      for r in schema.returns)
```

Every ATen operator carries a `_schema` — a structured signature description. Among other things, the schema says, for each return value, whether it *aliases* one of the inputs (i.e. shares storage with it). We check `r.alias_info is not None` on each return; if any return aliases anything, the whole op is “aliasing” and contributes zero new bytes.

Op	Why it aliases	alias_in_schema
<code>aten.relu_(x)</code>	in-place: writes into <code>x</code> , returns <code>x</code>	True
<code>aten.view(x, shape)</code>	different-shape view of <code>x</code> 's storage	True
<code>aten.transpose(x, ...)</code>	view with permuted strides	True
<code>aten._foreach_add_(params, grads)</code>	in-place on <code>params</code>	True
<code>aten.split(x, sizes)</code>	each output is a slice/view of <code>x</code>	True
<code>aten._fused_adam(params, grads, m, v, ...)</code>	in-place on <code>params</code> , <code>m</code> , <code>v</code>	True
<code>aten.matmul(a, b)</code>	allocates a fresh output tensor	False

Op	Why it aliases	alias_in_schema
<code>aten.add(a, b)</code>	allocates a fresh output	False
<code>operator.getitem</code>	no <code>_schema</code> at all → falls through	False

Without this check we’d double-count: a chain like `y = relu(x); z = view(y, ...); w = transpose(z, 0, 1)` would count `x`’s bytes four times (once for `x`, plus once for each “alias” the walk treats as a fresh allocation). By marking aliasing nodes with `allocation_bytes = 0` and using `compute_aliases` to chain them all back to `x`’s storage, the bytes get counted exactly once over the union of all four lifetimes.

`alias_in_schema` is the schema-level ground truth that drives the entire aliasing logic. The other detectors below (e.g. `is_getitem_of_container`) handle cases where a node has no schema but the same “this is a reference, not an allocation” semantics apply.

Pattern B — getitem unpacking of a multi-output op `_foreach_div(a_list, b_list)` returns `List[Tensor]` of 62 elements on ResNet18; each element is then accessed by an `operator.getitem` node. There are two flavors and they need different handling:

- **Aliasing parent** (e.g. `aten.split` whose outputs view the input; `_foreach_mul` whose outputs alias the inputs in-place): each `getitem(parent, i)` should chain through to the *i-th* input element. `_alias_target` does this — for single-input ops (split) all getitems point to that input; for list-input ops (foreach) `getitem(parent, i)` points to `parent.args[0][i]`.
- **Independent parent** (e.g. `convolution_backward` returns three *new* tensors): each `getitem` owns *its own* element bytes, so the per-element role can be different (one element is GRAD, another is OTHER). We had to fight specifically for this — see §4.3.

Pattern C — placeholder persistence Already discussed in §4.1. Implementation-wise it’s just a branch in `memory_timeline_by_role` that sets `lo=0, hi=N-1` for placeholders.

The unifying mechanism `_compute_aliases` builds a single `storage_owner: Dict[Node, Node]` map. For every node it asks “who really owns my bytes?” and writes the answer. Then `memory_timeline_by_role` iterates by owner (each owner contributes its size exactly once across the right interval), and merges roles across all aliases of an owner using `max()` over `NodeType` (an `IntEnum` whose values double as priority: `PARAM > GRAD > ACT > OTHER`).

4.3 Per-element role attribution (the `conv_backward` trap)

This was the bug that ate the most time. `conv_backward` returns three independent tensors:

```
(grad_input, grad_weight, grad_bias) = convolution_backward(...)
```

`grad_input` is an *activation* gradient (it flows into the upstream conv’s backward op and is then freed). `grad_weight` and `grad_bias` are *parameter* gradients (they flow into the optimizer’s foreach call). By role they should split: `grad_input` → OTHER, `grad_weight/bias` → GRAD.

A first attempt at “always make `getitem` an alias of its parent” lumped all three under the parent’s owner. When we then merged roles across the parent’s aliases, the GRAD-classified `getitem`s forced the *whole* parent’s bytes (including the much larger `grad_input`) into the GRAD bucket. On ResNet18 `bs=32`, GRAD jumped from a correct ~44 MB to a wrong 593 MB.

The right policy (in `_allocation_bytes`):

- if the parent has `alias_info` on its returns → it’s a view-/in-place multi-output → `getitem` aliases the parent’s input;
- otherwise (parent allocates independent tensors) → each `getitem` owns its own element bytes and gets its own role.

After the fix, ResNet18 bs=32 GRAD is back at ~44 MB, the activation gradients live in OTHER where they belong, and the BERT optimizer chain (`denom = _foreach_sqrt(v); _foreach_mul_(denom, ...); _foreach_addcdiv(...)`) correctly counts the scratch buffer once.

5. Results

Running `python starter_code.py -b 4 8 16 32` produces 16 breakdown charts (4 models $\times$ 4 batch sizes) plus 4 peak-vs-batch charts (one per model). Headline numbers from a full sweep (peak = static FX walk):

Model	BS	Peak (MB)	Latency (ms)
dummy	4	1.9	282.9
dummy	8	2.3	294.8
dummy	16	2.9	310.2
dummy	32	4.1	281.1
resnet18	4	258.3	910.5
resnet18	8	331.5	961.4
resnet18	16	498.4	986.6
resnet18	32	832.2	1046.6
resnet50	4	653.0	2414.1
resnet50	8	972.3	2597.2
resnet50	16	1612.1	2630.3
resnet50	32	2915.7	2795.0
bert	4	2088.3	3447.7
bert	8	2088.3	3528.2
bert	16	2505.9	3792.7
bert	32	3739.3	3913.3

Reading the breakdown plots

For ResNet (the canonical case): - The **PARAM** band at the bottom is constant — model parameters persist all iteration. - The **green ACT hump** rises during forward (each layer’s output must stay alive until backward consumes it), peaks just before the forward/backward boundary, and falls during backward as activations are released. - The **orange GRAD band** appears during backward as parameter gradients are produced. - **OPT_STATE** is flat because Adam’s persistent state (`exp_avg`, `exp_avg_sq`, and step tensors) is allocated before profiling and remains live for the whole iteration. - **OTHER** (gray) contains temporary tensors that are not parameters, saved activations, parameter gradients, or persistent optimizer state. In the optimizer region this often forms a sawtooth pattern. PyTorch’s `foreach=True` Adam updates many tensors as lists, using ops such as `_foreach_mul`, `_foreach_addcmul`, `_foreach_sqrt`, `_foreach_div`, and `_foreach_addcdiv`. Some of these ops allocate a temporary tensor list; the next copy or in-place update consumes it. The repeated allocate-then-consume pattern makes gray rise and fall. This is real optimizer scratch memory, but it is short-lived and separate from the flat **OPT_STATE** band.

For BERT, peak shifts: at small batch the peak is in the optimizer region (Adam moments dominate); at large batch the activation hump in the forward/backward region exceeds it.

For dummy MLP, the model is too small for activations to dominate; the chart is mostly the constant parameter and optimizer-state baseline. The bar chart is essentially flat — useful as a profiler-correctness check, less useful for showing batch-size scaling.

Reading the peak-vs-batch plots

ResNet18 / ResNet50 / BERT all scale roughly linearly with batch size once the activation hump dominates. Dummy doesn’t scale because it has no meaningful activation memory at this size.

6. Known limitations

- **Workspace memory is invisible.** cuDNN/cuBLAS scratch and other op-internal allocations are real GPU bytes but aren't outputs of FX nodes, so the static walk doesn't include them. Typically 10–50 MB on ResNet, ~9 MB on dummy — small relative to the activation hump.
- **Static walk can over-extend lifetime in foreach in-place chains** if the order of `parent.args[0]` doesn't match the output ordering — we trust that `output[i]` aliases `args[0][i]`, which is true for the ATen `_foreach_*` family but not formally guaranteed for every op with `alias_info`.
- **Multi-output ops with mixed users** (some users are `getitems`, some are not) keep the parent's full bytes; this case is rare in the models we tested but isn't perfectly handled.

7. Files

File	Purpose
starter_code.py	Phase 1 entry point — sweep (<code>model</code> , <code>batch_size</code>), save plots
graph_prof.py	the <code>GraphProfiler</code> and the static analysis described above
models.py	factory: dummy MLP, ResNet18/50, BERT-base + Adam(<code>foreach=True</code>)
visualizer.py	the two plot helpers
graph_tracer.py	provided — <code>compile()</code> + <code>SEPFunction</code>
utils.py	provided — <code>SPMD_DECOMP_TABLE</code> decomposition rules

To run:

```
python starter_code.py                                # everything
python starter_code.py resnet18 -b 8 16 32            # one model, three batch sizes
```